République du Cameroun
Paix – Travail – Patrie

Université de Yaoundé I

École Nationale Supérieure
Polytechnique de Yaoundé
(ENSPY)

Republic of Cameroon
Peace – Work – Fatherland

University of Yaoundé I

National Advanced School of
Engineering of Yaoundé
(NASEY)

Rapport Académique

Projet Base de Données

Talky

« Parlez sans frontières »

Conception et Implémentation d'une Application
de Messagerie Instantanée et d'Appels Temps Réel

Membres du groupe :

1. **ETCHOME Chris Manuel Loïc**
2. **DOUANLA FOKOU Ange Kendra**
3. **ABEDE NDZANA Anatole**
4. **KAZITI DIZEKETI Simon Pierre**
5. **FOALEM SIMO Alfred**
6. **BENAE MPEKE Blaise**

Sous la supervision de :

Dr NANA MBINKEU

Année académique 2025 - 2026

À nos familles, sources d'inspiration et de soutien indéfectible.

À notre encadrant, Dr NANA MBINKEU,
pour sa guidance éclairée tout au long de ce projet.

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Nos premiers remerciements vont au **Dr NANA MBINKEU**, notre encadrant, dont les conseils avisés, la disponibilité et les retours constructifs ont été déterminants dans l'orientation et la réussite de ce travail.

Nous remercions également le corps enseignant du Département de Génie Informatique de l'**École Nationale Supérieure Polytechnique de Yaoundé (ENSPY)** pour la qualité de la formation dispensée, qui nous a fourni les bases théoriques et pratiques nécessaires à ce projet.

Enfin, nous remercions l'ensemble de nos pairs et camarades de promotion pour les échanges fructueux et l'entraide permanente tout au long de ce cycle de formation.

Résumé

Ce rapport présente la conception et l'implémentation de **Talky**, une application mobile de messagerie instantanée et d'appels audio/vidéo temps réel, développée dans le cadre du module *Projet Base de Données* à l'École Nationale Supérieure Polytechnique de Yaoundé.

Talky cible les usages professionnels internes à l'entreprise et offre, à l'instar de WhatsApp, un ensemble riche de fonctionnalités : messagerie texte et multimédia, appels individuels et de groupe via WebRTC, publication de statuts éphémères de 24 heures, gestion des contacts, historique des appels et personnalisation de l'interface.

L'application repose sur une architecture client-serveur hybride : **Flutter + Riverpod** côté client mobile, **Firebase** (Firestore, FCM, Realtime Database) pour la persistance et les notifications, **Cloudinary** pour le stockage des médias, et un serveur de signalisation **Node.js/Socket.IO** déployé sur Render.com pour la coordination des appels WebRTC.

Le document couvre l'ensemble des phases du cycle de développement : analyse du problème, étude de l'existant, modélisation des données (MCD, MLD), modélisation UML, architecture technique, implémentation et perspectives.

Mots-clés : messagerie instantanée, WebRTC, Flutter, Firebase, Firestore, Socket.IO, Cloudinary, temps réel, NoSQL, application mobile.

Abstract — This report presents the design and implementation of *Talky*, a mobile instant-messaging and real-time audio/video calling application developed at the National Advanced School of Engineering of Yaoundé. The system leverages Flutter, Firebase, WebRTC and a Node.js signalling server. Keywords : instant messaging, WebRTC, Flutter, Firebase, real-time, NoSQL.

Table des matières

Remerciements **2**

Résumé **3**

Liste des abréviations **9**

1 Introduction générale **10**
1.1 Contexte 10
1.2 Problématique 10
1.3 Objectifs du projet 11
1.3.1 Objectif général 11
1.3.2 Objectifs spécifiques 11
1.4 Structure du rapport 11

2 Analyse et étude de l'existant **12**
2.1 Panorama des solutions existantes 12
2.1.1 WhatsApp 12
2.1.2 Telegram 12
2.1.3 Signal 12
2.1.4 Microsoft Teams / Slack 12
2.2 Tableau comparatif 13
2.3 Positionnement de Talky 13
2.4 Analyse des besoins 13
2.4.1 Besoins fonctionnels 13
2.4.2 Besoins non fonctionnels 14

3 Modélisation des données **15**
3.1 Présentation du système de stockage 15
3.2 Modèle Conceptuel des Données (MCD) 15
3.2.1 Description des entités 16
3.3 Modèle Logique des Données (MLD) 17

3.4 Modèle physique — Index Firestore . . . 17
3.5 Modélisation mathématique . . . 18
3.5.1 Complexité des requêtes Firestore . . . 18
3.5.2 Modèle d'expiration des statuts . . . 18
3.5.3 Latence d'un appel WebRTC . . . 18
3.5.4 Présence en temps réel — Firebase Realtime Database . . . 19

4 Modélisation UML **20**
4.1 Diagramme des cas d'utilisation . . . 20
4.2 Diagramme de classes . . . 21
4.3 Diagramme de séquence — Envoi d'un message . . . 22
4.4 Diagramme de séquence — Appel WebRTC . . . 23
4.5 Diagramme d'états — Cycle de vie d'un message . . . 24
4.6 Diagramme d'activité — Authentification . . . 25

5 Architecture technique **26**
5.1 Vue d'ensemble . . . 26
5.2 Architecture Flutter — Feature-First . . . 27
5.3 Serveur de signalisation WebRTC . . . 27
5.4 Système de notifications push . . . 28

6 Implémentation **29**
6.1 Environnement et stack technologique . . . 29
6.2 Fonctionnalités implémentées . . . 29
6.2.1 Authentification par OTP . . . 30
6.2.2 Messagerie instantanée . . . 30
6.2.3 Appels audio/vidéo — WebRTC . . . 30
6.2.4 Statuts éphémères . . . 31
6.2.5 Gestion de la présence . . . 31
6.3 Interfaces utilisateur . . . 31
6.4 Sécurité . . . 32

7 Conclusion et perspectives **33**
7.1 Bilan . . . 33
7.2 Difficultés rencontrées . . . 33
7.3 Perspectives d'évolution . . . 34

A Structure du projet Flutter **36**

B Serveur de signalisation — Extraits clés **38**

C Règles de sécurité Firestore **39**

Table des figures

3.1 Modèle Conceptuel des Données (MCD) de Talky 16

4.1 Diagramme des cas d'utilisation de Talky 20
4.2 Diagramme de classes de Talky 21
4.3 Diagramme de séquence — Envoi d'un message texte 22
4.4 Diagramme de séquence — Établissement d'un appel WebRTC 23
4.5 Diagramme d'états — Cycle de vie d'un message 24
4.6 Diagramme d'activité — Processus d'authentification 25

5.1 Architecture globale de Talky 26

Liste des tableaux

2.1 Comparaison des principales applications de messagerie 13

3.1 Entités et attributs principaux 16
3.2 Types de messages supportés (`MessageType`) 17
3.3 Index composite Firestore sur `conversations` 18

5.1 Événements Socket.IO du serveur de signalisation 28

6.1 Stack technologique de Talky 29
6.2 Écrans principaux de l'application Talky 31

Liste des abréviations

Sigle	Signification
API	Application Programming Interface
BDD	Base De Données
CDN	Content Delivery Network
FCM	Firebase Cloud Messaging
GI	Génie Informatique
ICE	Interactive Connectivity Establishment
JSON	JavaScript Object Notation
MCD	Modèle Conceptuel des Données
MLD	Modèle Logique des Données
NoSQL	Not Only SQL
OTP	One-Time Password
P2P	Peer-to-Peer
SDP	Session Description Protocol
SGBD	Système de Gestion de Bases de Données
STUN	Session Traversal Utilities for NAT
TURN	Traversal Using Relays around NAT
UML	Unified Modeling Language
VoIP	Voice over Internet Protocol
WebRTC	Web Real-Time Communication

Chapitre 1

Introduction générale

1.1 Contexte

La communication numérique a radicalement transformé la manière dont les individus et les organisations échangent de l'information. Les applications de messagerie instantanée — WhatsApp, Telegram, Signal, Microsoft Teams — sont désormais au cœur des pratiques professionnelles et personnelles à l'échelle mondiale. En Afrique subsaharienne, et particulièrement au Cameroun, la pénétration des smartphones et l'essor des réseaux mobiles 4G ont créé un besoin croissant de solutions de communication rapides, fiables et économiques.

Dans le cadre du module *Projet Base de Données* du Département de Génie Informatique de l'ENSPY, notre groupe a été chargé de concevoir et d'implémenter une solution logicielle complète répondant à des besoins réels de communication. Nous avons choisi de développer **Talky**, une application de messagerie instantanée et d'appels temps réel pensée pour un usage professionnel interne.

1.2 Problématique

Les entreprises et organisations camerounaises utilisent majoritairement des solutions propriétaires étrangères pour leurs communications internes. Ces solutions présentent plusieurs inconvénients majeurs :

- **Dépendance aux serveurs étrangers :** les données des employés transitent par des infrastructures hors du contrôle de l'organisation.
- **Absence de personnalisation :** impossibilité d'adapter l'outil aux processus internes ou à l'identité visuelle de l'entreprise.
- **Coûts de licence :** les solutions professionnelles comme Teams ou Zoom impliquent des abonnements onéreux.
- **Problèmes de confidentialité :** la collecte de métadonnées par les grandes plateformes représente un risque pour la confidentialité des échanges.

Question centrale

Comment concevoir et implémenter une application mobile de communication interne à la fois performante, sécurisée, économique et adaptable, tout en s'appuyant sur des technologies modernes accessibles à une équipe étudiante ?

1.3 Objectifs du projet

1.3.1 Objectif général

Concevoir et développer une application mobile multiplateforme de messagerie instantanée et d'appels audio/vidéo temps réel, destinée à un usage professionnel interne.

1.3.2 Objectifs spécifiques

1. Implémenter un système de messagerie textuelle et multimédia avec accusés de réception.
2. Intégrer des appels audio et vidéo individuels et de groupe via WebRTC.
3. Développer un système de statuts éphémères (24 heures) similaire aux Stories.
4. Assurer les notifications push en temps réel via Firebase Cloud Messaging.
5. Proposer une interface utilisateur intuitive, moderne et accessible.
6. Mettre en place une authentification robuste par OTP.

1.4 Structure du rapport

Le rapport s'articule en sept chapitres. Le chapitre 2 présente l'étude de l'existant. Le chapitre 3 couvre la modélisation des données (MCD/MLD). Le chapitre 4 présente la modélisation UML. Le chapitre 5 décrit l'architecture technique. Le chapitre 6 détaille l'implémentation. Le chapitre 7 conclut et ouvre des perspectives.

Chapitre 2

Analyse et étude de l'existant

2.1 Panorama des solutions existantes

2.1.1 WhatsApp

WhatsApp Messenger, propriété de Meta, est l'application de messagerie la plus utilisée au monde avec plus de 2,7 milliards d'utilisateurs actifs. Elle offre messagerie texte et multimédia, appels audio/vidéo, statuts de 24 heures et chiffrement de bout en bout. Son principal inconvénient pour un usage professionnel reste l'absence de contrôle sur les données et d'impossibilité de personnalisation.

2.1.2 Telegram

Telegram se distingue par ses groupes de grande capacité (jusqu'à 200 000 membres), ses canaux de diffusion, la prise en charge de bots et une API ouverte. Il propose également un mode Secret avec chiffrement de bout en bout et auto-destruction des messages.

2.1.3 Signal

Signal est la référence en matière de confidentialité, avec un chiffrement de bout en bout basé sur son propre protocole. Il reste toutefois limité en fonctionnalités professionnelles avancées.

2.1.4 Microsoft Teams / Slack

Ces solutions professionnelles offrent des fonctionnalités de collaboration avancées (partage de fichiers, réunions, intégrations tierces) mais impliquent des coûts significatifs et une infrastructure complexe.

2.2 Tableau comparatif

TABLE 2.1 – Comparaison des principales applications de messagerie

Critère	WhatsApp	Telegram	Signal	Teams	Talky
Messagerie texte	✓	✓	✓	✓	✓
Médias	✓	✓	✓	✓	✓
Appels VoIP	✓	✓	✓	✓	✓
Statuts éphémères	✓	✓	×	×	✓
Open source	×	Partiel	✓	×	✓
Mode confidentiel	Limité	✓	✓	×	Prévu
Personnalisable	×	×	×	Limité	✓
Données maîtrisées	×	×	Partiel	×	✓

2.3 Positionnement de Talky

Valeur ajoutée de Talky

- **Maîtrise des données :** l'organisation déploie son propre serveur de signalisation et contrôle intégralement la base Firebase.
- **Mode fantôme (Ghost Mode) :** l'utilisateur peut masquer son statut de connexion sans quitter l'application.
- **Messages confidentiels :** auto-destruction configurable de 5 secondes à une durée personnalisée (fonctionnalité planifiée).
- **Traduction instantanée intégrée :** prévu pour la version 2.0.
- **Architecture extensible :** base Flutter multiplateforme (Android, iOS, Web).

2.4 Analyse des besoins

2.4.1 Besoins fonctionnels

BF1 Inscription via numéro de téléphone avec vérification OTP.

BF2 Envoi/réception de messages texte, images, vidéos, audios et fichiers.

BF3 Appels audio et vidéo individuels et de groupe.

BF4 Publication de statuts éphémères visibles 24 heures.

BF5 Création et gestion de groupes de discussion.

BF6 Notifications push même lorsque l'application est fermée.

BF7 Archivage, épinglage et suppression de conversations.

BF8 Affichage du statut de présence des contacts.

2.4.2 Besoins non fonctionnels

BNF1 **Performance :** latence des messages inférieure à 500 ms.

BNF2 **Disponibilité :** serveur opérationnel 24h/24, 7j/7.

BNF3 **Sécurité :** tokens et clés API non exposés dans le code source.

BNF4 **Scalabilité :** groupes jusqu'à 256 membres.

BNF5 **Maintenabilité :** architecture modulaire par fonctionnalités.

BNF6 **Compatibilité :** Android API 21+ (Android 5.0 minimum).

Chapitre 3

Modélisation des données

3.1 Présentation du système de stockage

Talky repose sur une architecture de données **NoSQL distribuée** combinant Firebase Firestore (persistance principale) et Firebase Realtime Database (présence temps réel).

Justification du choix NoSQL

Le recours à Firestore plutôt qu'un SGBD relationnel est motivé par : (1) la synchronisation temps réel sans polling, (2) la flexibilité du schéma pour les messages hétérogènes, (3) l'intégration native avec Firebase Auth et FCM, et (4) la scalabilité horizontale automatique.

3.2 Modèle Conceptuel des Données (MCD)

Le MCD ci-dessous est établi selon la méthode MERISE. Il représente les entités du système et leurs associations.

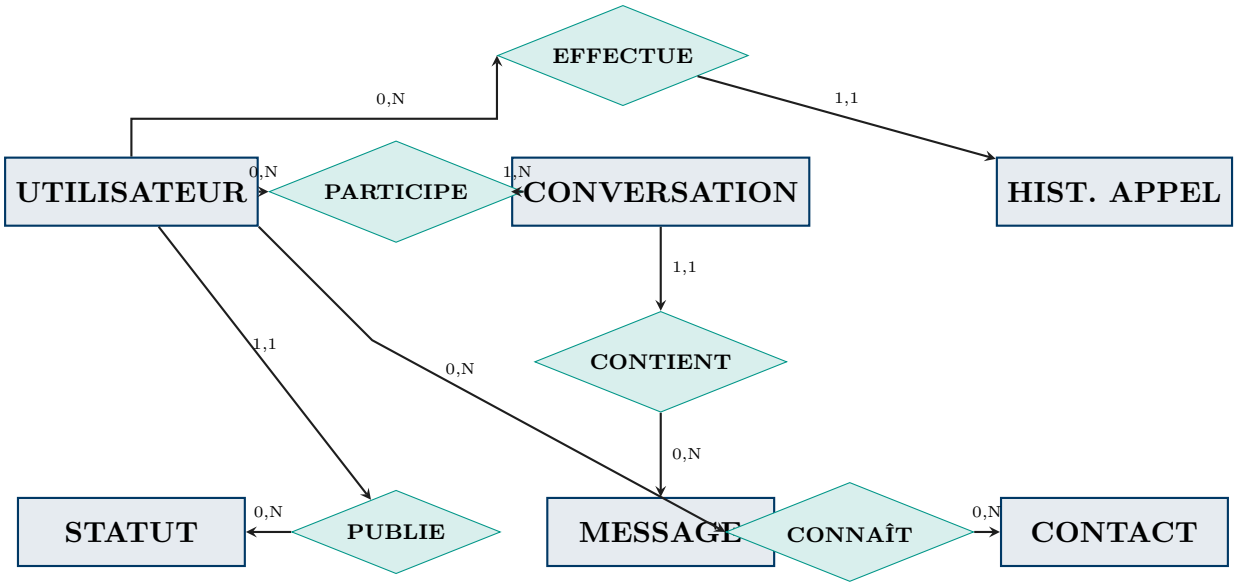

FIGURE 3.1 – Modèle Conceptuel des Données (MCD) de Talky

3.2.1 Description des entités

TABLE 3.1 – Entités et attributs principaux

Entité	Clé primaire	Attributs principaux
UTILISATEUR	uid (String)	name, phone, email, photoUrl, status, isOnline, lastSeen, ghostMode, preferredLanguage
CONVERSATION	id (String)	participantIds, isGroup, groupName, groupPhoto, lastMessage, lastMessageAt, isPinned, isArchived, unreadCount
MESSAGE	id (String)	conversationId, senderId, content, type, status, sentAt, readAt, mediaUrl, isDeleted, isEdited, replyToId
STATUT	id (String)	userId, type, text, mediaUrl, backgroundColor, createdAt, expiresAt, viewedBy, likedBy
HIST. APPEL	id (String)	callerId, receiverId, type (entrant/sortant/manqué), isVideo, durationSeconds, timestamp
CONTACT	id (String)	contactName, contactPhoto, phoneNumber, addedAt

3.3 Modèle Logique des Données (MLD)

Le MLD traduit le MCD en collections Firestore. Notation : *Collection(clé, attribut, cle_etrangere^{réf})*.

MLD — Collections Firestore

users(uid, name, phone, email, photoUrl, status, preferredLanguage, isOnline, lastSeen, ghostMode, fcmToken)

conversations(id, participantIds[], participantNames{uid → name}, participantPhotos{uid → url}, lastMessage, lastMessageSenderId$^{\rightarrow users}$, lastMessageType, lastMessageStatus, lastMessageAt, unreadCount{uid → int}, isGroup, groupName, groupPhoto, isPinned, isArchived)

conversations/{id}/messages(id, conversationId$^{\rightarrow conversations}$, senderId$^{\rightarrow users}$, senderName, content, type, status, sentAt, readAt, mediaUrl, mediaName, mediaDuration, replyToId$^{\rightarrow messages}$, isStatusReply, isDeleted, isEdited, editedAt, deletedFor[])

statuses(id, userId$^{\rightarrow users}$, userName, userPhoto, type, text, mediaUrl, backgroundColor, createdAt, expiresAt, viewedBy[], viewedAt{uid → ts}, likedBy[])

users/{uid}/call_history(id, callerId$^{\rightarrow users}$, callerName, receiverId$^{\rightarrow users}$, receiverName, isGroup, groupName, participantIds[], type, timestamp, durationSeconds, isVideo)

users/{uid}/contacts(id, contactName, contactPhoto, phoneNumber, addedAt)

Table 3.2 – Types de messages supportés (`MessageType`)

Valeur	Description	Champs additionnels
`text`	Message texte pur	content
`image`	Image (JPEG/PNG/GIF)	mediaUrl, mediaName
`video`	Vidéo	mediaUrl, mediaName, mediaDuration
`audio`	Note vocale	mediaUrl, mediaDuration
`file`	Document/fichier	mediaUrl, mediaName
`deleted`	Message supprimé (tombstone)	isDeleted = true

3.4 Modèle physique — Index Firestore

Pour garantir les performances, un index composite est défini sur la collection `conversations` :

TABLE 3.3 – Index composite Firestore sur `conversations`

Champ	Type	Ordre	Justification
participantIds	Arrays	—	Filtrage par utilisateur
isArchived	Ascending	Croissant	Séparation actif/archivé
isPinned	Descending	Décroissant	Épinglés en tête
lastMessageAt	Descending	Décroissant	Tri chronologique inversé

3.5 Modélisation mathématique

3.5.1 Complexité des requêtes Firestore

Soit C le nombre total de conversations et k le nombre de résultats retournés. Avec l'index composite, la complexité d'une requête de listing est :

$$T_{\text{query}} = O(\log C + k)$$

Sans index, elle serait $O(C)$ (scan complet), ce qui deviendrait inacceptable à grande échelle.

3.5.2 Modèle d'expiration des statuts

Un statut s est *valide* à l'instant t (en secondes Unix) si et seulement si :

$$t_{\text{created}}(s) \;\leq\; t \;\leq\; t_{\text{created}}(s) + 86400$$

Le taux de vues d'un statut est :

$$\tau_{\text{view}}(s) = \frac{|\,viewedBy(s)\,|}{|\,contacts(u)\,|} \times 100 \quad (\%)$$

3.5.3 Latence d'un appel WebRTC

La latence totale perçue est la somme de trois composantes :

$$L_{\text{total}} = L_{\text{signaling}} + L_{\text{ICE}} + L_{\text{media}}$$

- $L_{\text{signaling}} \approx 50$–$200$ ms (Socket.IO sur Render.com)
- $L_{\text{ICE}} \approx 100$–$500$ ms (négociation STUN/TURN)
- $L_{\text{media}} < 150$ ms (recommandation ITU-T G.114 pour la voix)

3.5.4 Présence en temps réel — Firebase Realtime Database

La structure JSON utilisée pour la présence est :

Listing 3.1 – Structure Realtime Database pour la présence

```
{
  "presence": {
    "{userId}": {
      "isOnline": true,
      "lastSeen": 1704067200000   // timestamp Unix ms
    }
  }
}
```

Chapitre 4

Modélisation UML

4.1 Diagramme des cas d'utilisation

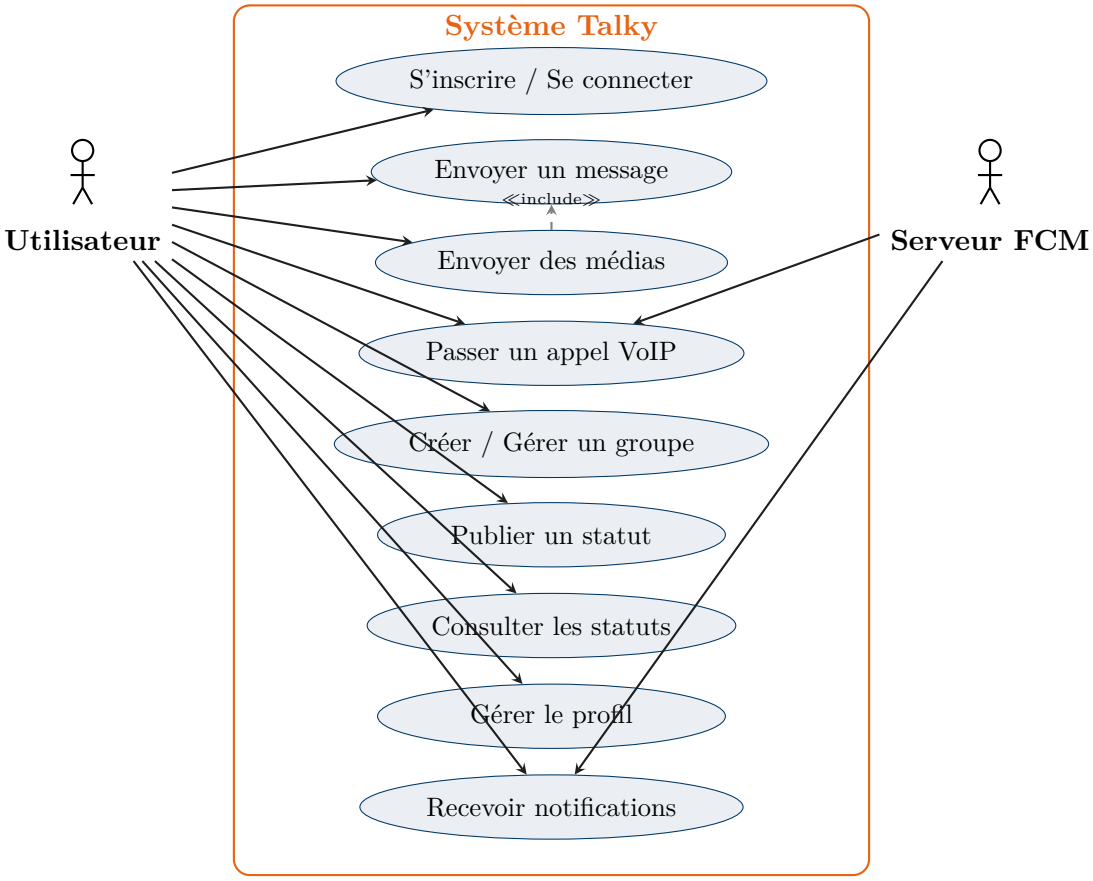

FIGURE 4.1 – Diagramme des cas d'utilisation de Talky

4.2 Diagramme de classes

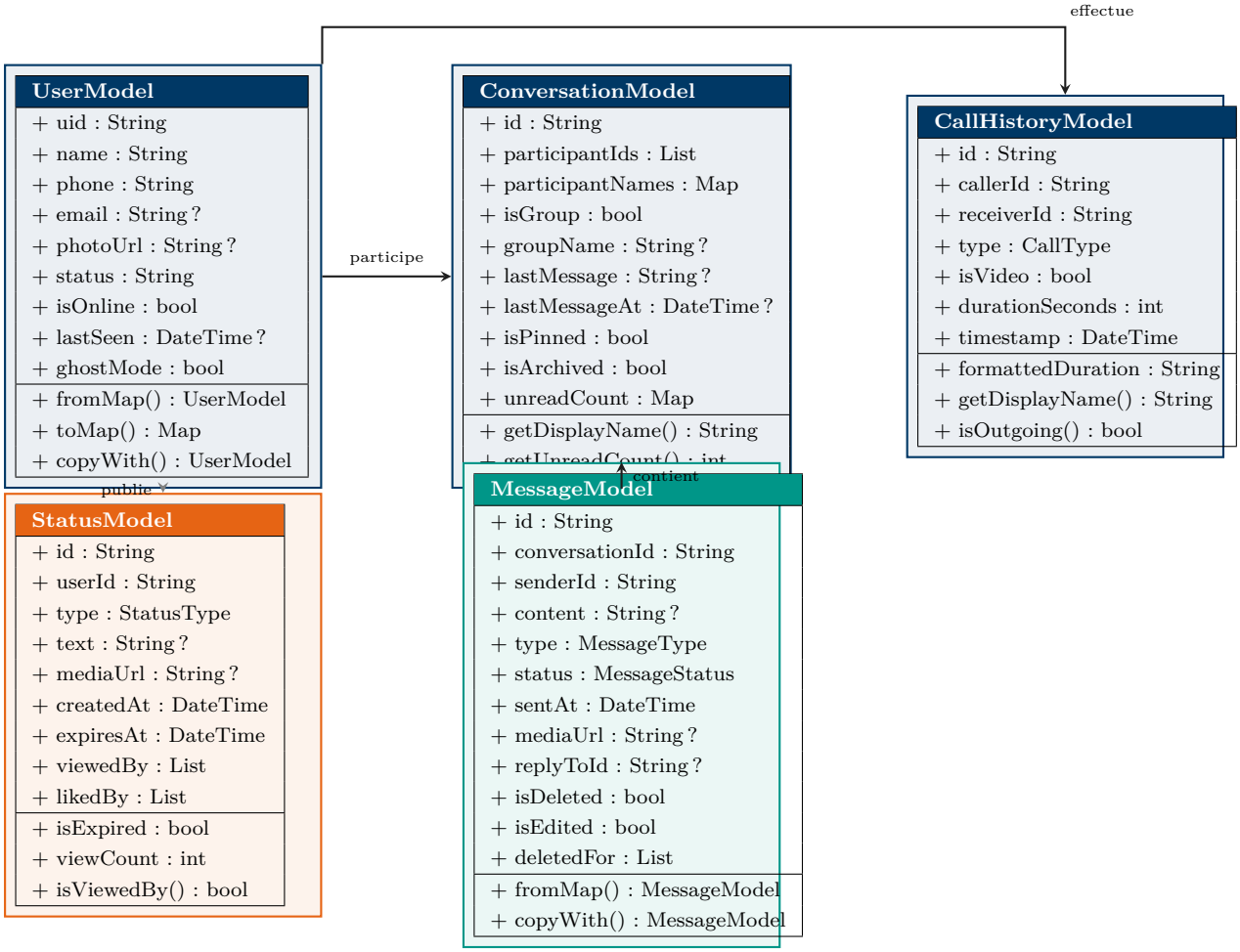

FIGURE 4.2 – Diagramme de classes de Talky

4.3 Diagramme de séquence — Envoi d'un message

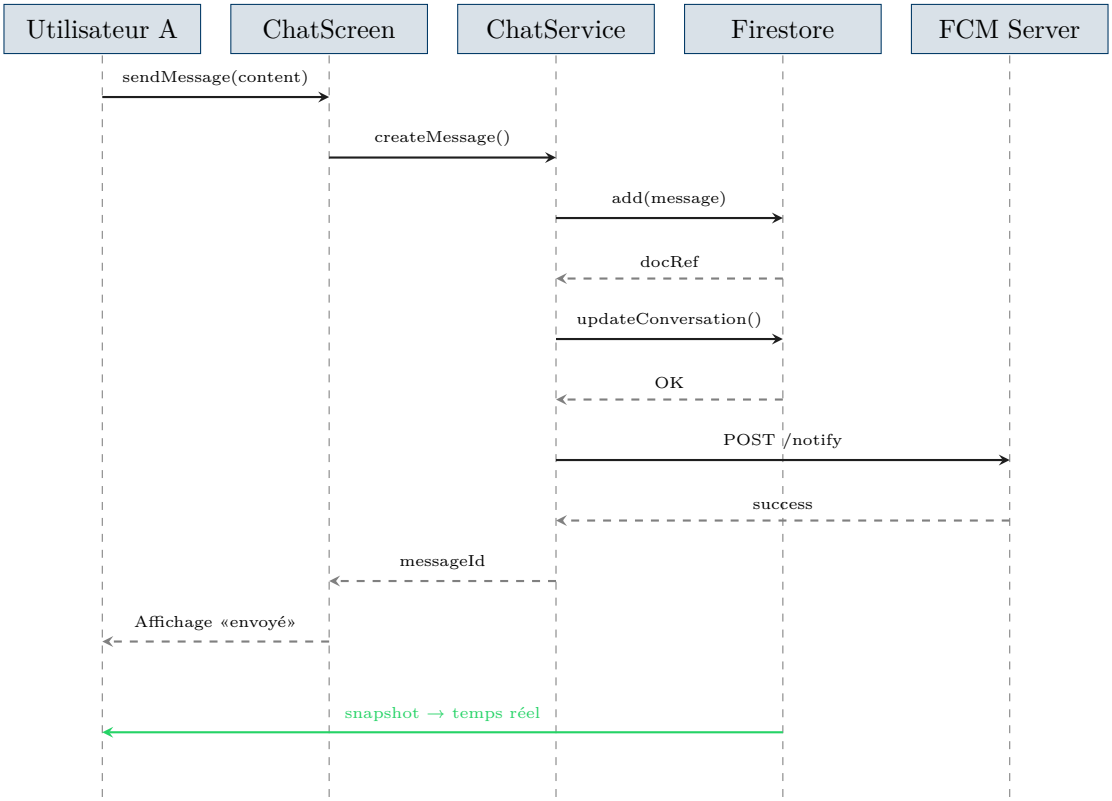

FIGURE 4.3 – Diagramme de séquence — Envoi d'un message texte

4.4 Diagramme de séquence — Appel WebRTC

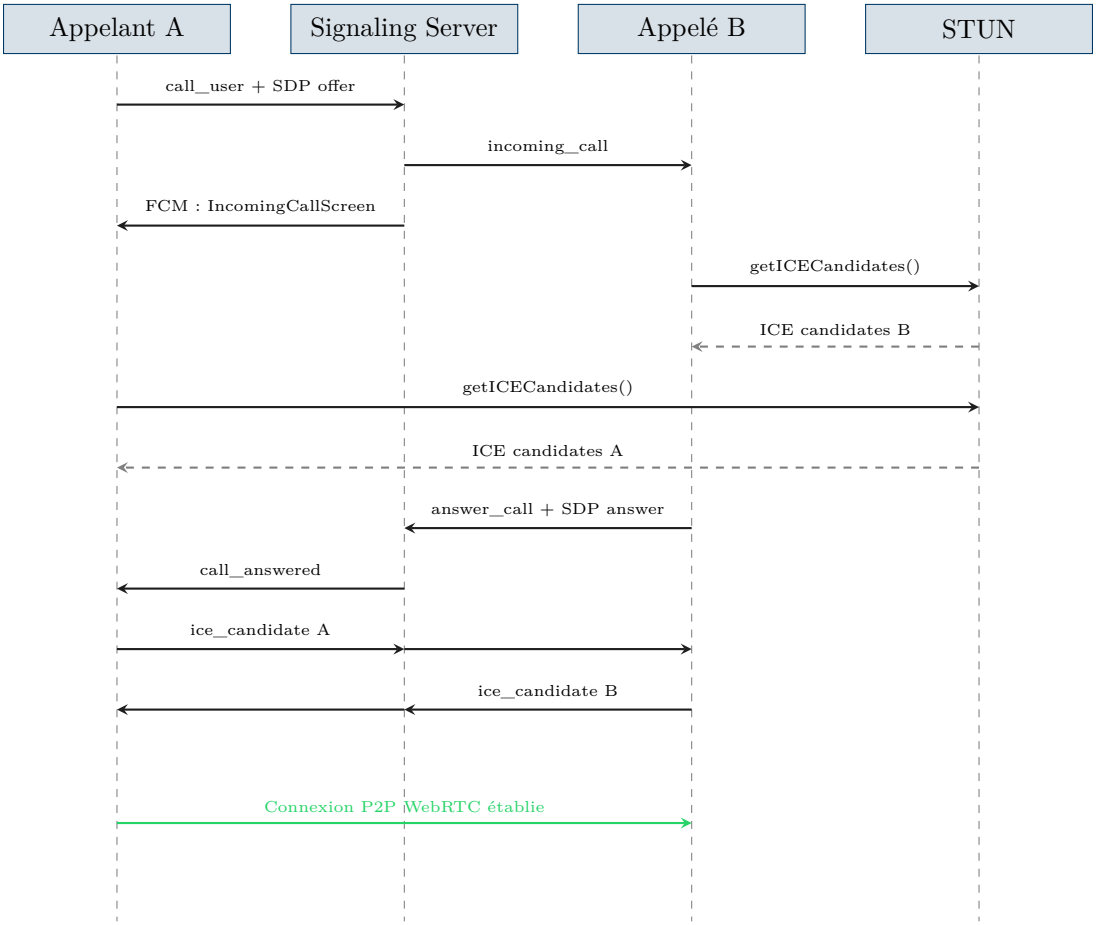

FIGURE 4.4 – Diagramme de séquence — Établissement d'un appel WebRTC

4.5 Diagramme d'états — Cycle de vie d'un message

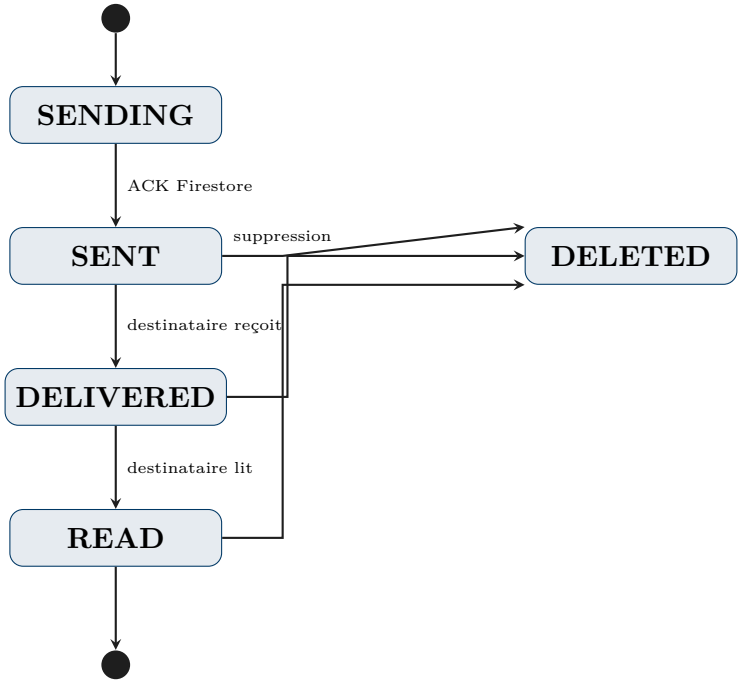

FIGURE 4.5 – Diagramme d'états — Cycle de vie d'un message

4.6 Diagramme d'activité — Authentification

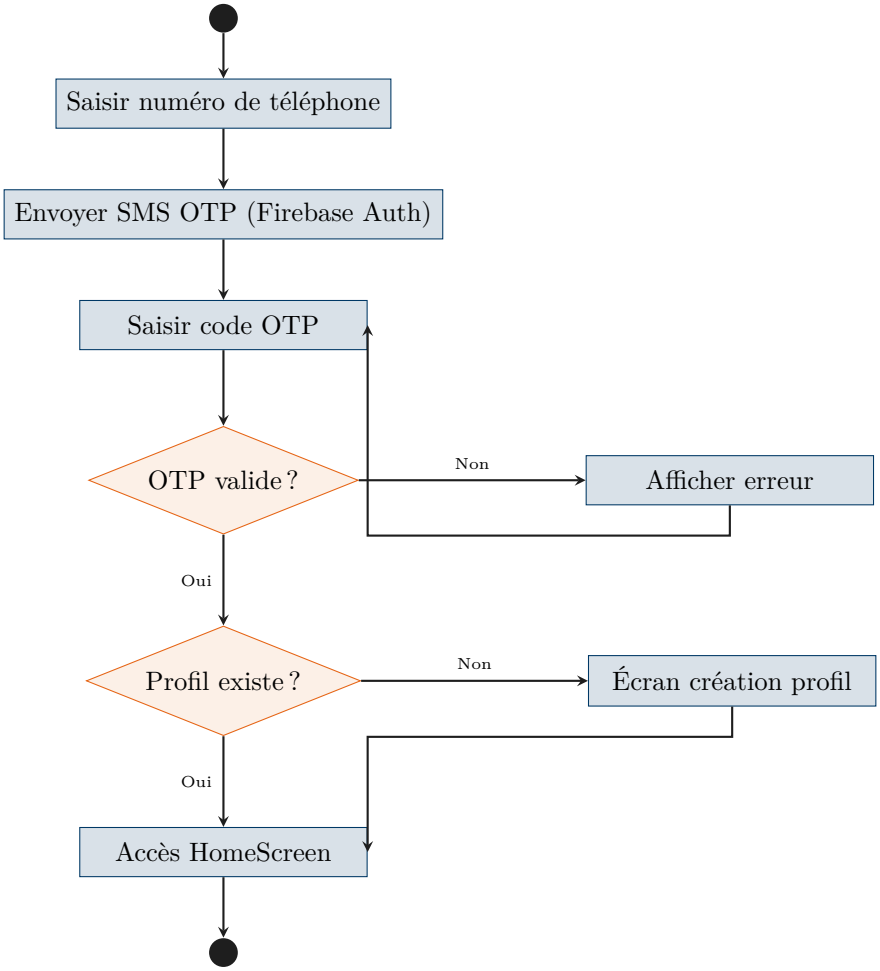

FIGURE 4.6 – Diagramme d'activité — Processus d'authentification

Chapitre 5

Architecture technique

5.1 Vue d'ensemble

Talky adopte une architecture **client-serveur hybride** combinant :

— Un client mobile Flutter avec gestion d'état Riverpod (architecture Feature-First).

— Une couche BaaS Firebase pour la persistance et les notifications.

— Un serveur de signalisation Node.js/Socket.IO pour la coordination WebRTC.

— Un CDN Cloudinary pour le stockage et la transformation des médias.

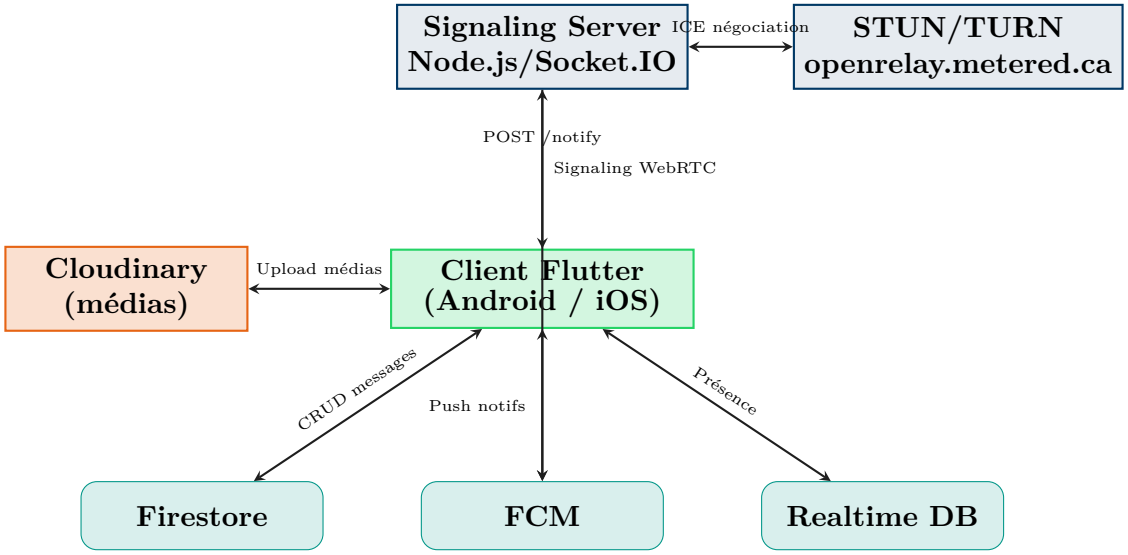

FIGURE 5.1 – Architecture globale de Talky

5.2 Architecture Flutter — Feature-First

Structure modulaire du projet Flutter

`lib/`

- `core/` — Services transverses : notifications, présence, routage, thème, cache
- `features/auth/` — Inscription, OTP, gestion de session Firebase
- `features/chat/` — Messagerie, médias, conversations, groupes
- `features/calls/` — Appels WebRTC audio/vidéo (individuel et groupe)
- `features/status/` — Statuts éphémères 24h
- `features/settings/` — Paramètres, thème, langue, profil
- `shared/` — Widgets réutilisables (`TalkyButton`, `TalkyTextField`)

Chaque fonctionnalité est découpée en trois sous-couches :

1. `domain/` — Modèles de données purs (classes Dart sans dépendances Firebase).
2. `data/` — Services (accès Firestore, Socket.IO) et providers Riverpod.
3. `presentation/` — Écrans et widgets Flutter.

5.3 Serveur de signalisation WebRTC

Le serveur Node.js/Socket.IO assume deux rôles :

1. **Broker de signalisation :** acheminement des offres SDP, réponses SDP et candidats ICE.
2. **Gateway FCM :** endpoint REST `POST /notify` envoyant des messages *data-only*.

Table 5.1 – Événements Socket.IO du serveur de signalisation

Événement	Direction	Description
`register`	Client → Serveur	Enregistrement socket de l'utilisateur
`call_user`	Client → Serveur	Initiation d'appel + SDP offer
`incoming_call`	Serveur → Client	Notification d'appel entrant
`answer_call`	Client → Serveur	SDP answer de l'appelé
`call_answered`	Serveur → Caller	Transmission SDP answer à l'appelant
`reject_call`	Client → Serveur	Refus de l'appel par le destinataire
`ice_candidate`	Bidirectionnel	Échange de candidats ICE
`end_call`	Client → Serveur	Terminaison de l'appel
`create_group_call`	Client → Serveur	Création d'une salle d'appel groupe
`join_group_call`	Client → Serveur	Rejoindre une salle existante

5.4 Système de notifications push

Point clé — Notifications data-only

Les notifications FCM de Talky sont envoyées en mode **data-only** (sans champ `notification` dans le payload). Ce choix est fondamental : sur Android, la présence d'un champ `notification` entraîne la prise en main par le système Android, contournant Flutter et empêchant la navigation vers l'écran d'appel entrant. Avec un payload *data-only*, Flutter reçoit le message via `onMessage`/`onMessageOpenedApp` et contrôle lui-même l'affichage via `flutter_local_notifications`.

Chapitre 6

Implémentation

6.1 Environnement et stack technologique

Table 6.1 – Stack technologique de Talky

Composant	Technologie	Détail
Langage client	Dart / Flutter	Flutter 3.x, Dart 3.x
Gestion d'état	Riverpod	flutter_riverpod
Routage	GoRouter	go_router
Base de données	Firebase Firestore	cloud_firestore
Authentification	Firebase Auth	OTP SMS
Notifications	Firebase FCM	Messages data-only
Présence	Firebase Realtime Database	firebase_database
Médias	Cloudinary	cloud : `dvnxsn73m`
Appels VoIP	WebRTC	flutter_webrtc
Signalisation	Socket.IO	socket_io_client
Sonneries	flutter_ringtone_player	v4.0.0
Serveur	Node.js / Express	Render.com (free tier)
IDE	Visual Studio Code / Android Studio	—
Appareil de test	Redmi 14C	Android 15, API 35

6.2 Fonctionnalités implémentées

6.2.1 Authentification par OTP

L'authentification utilise Firebase Phone Authentication. L'utilisateur saisit son numéro international, reçoit un SMS de vérification, entre le code à 6 chiffres, puis complète son profil (nom, photo). Le routage est géré par GoRouter avec redirection automatique selon l'état d'authentification et la complétude du profil Firestore.

6.2.2 Messagerie instantanée

La messagerie est implémentée avec Firestore en temps réel (listeners `snapshots()`). Les fonctionnalités comprennent :

- Messages texte, images, vidéos, audio et fichiers.
- Réponse contextuelle à un message avec prévisualisation (reply).
- Modification et suppression de messages (pour tous ou pour soi).
- Accusés de réception : envoyé → reçu → lu (double coche bleue).
- Épinglage et archivage de conversations.
- Groupes jusqu'à 256 membres.

6.2.3 Appels audio/vidéo — WebRTC

Les appels P2P utilisent le protocole WebRTC avec le serveur STUN/TURN `openrelay.metered.ca`. La signalisation est gérée via Socket.IO. Les appels de groupe utilisent une architecture *mesh* complète.

Listing 6.1 – Création de la RTCPeerConnection — call_service.dart

```
Future<void> _createPeerConnection() async {
  final config = {
    'iceServers': [
      {'urls': 'stun:openrelay.metered.ca:80'},
      {
        'urls':       'turn:openrelay.metered.ca:80',
        'username':   'openrelayproject',
        'credential': 'openrelayproject',
      },
    ],
  };
  _peerConnection = await createPeerConnection(config);
  _peerConnection!.onIceCandidate = (candidate) {
    _socket!.emit('ice_candidate', {
      'targetUserId': _remoteUserId,
      'candidate':    candidate.toMap(),
    });
  };
```

```
}
```

6.2.4 Statuts éphémères

Les statuts (texte avec couleur de fond, image, vidéo) expirent automatiquement après 24 heures. Le système enregistre les vues et les likes. Une requête Firestore filtre les documents dont `expiresAt > now()`.

6.2.5 Gestion de la présence

La présence en temps réel utilise Firebase Realtime Database avec la fonctionnalité `onDisconnect()` pour mettre à jour automatiquement le statut hors-ligne lors d'une déconnexion, y compris inopinée. Le **Ghost Mode** permet de masquer sa présence.

6.3 Interfaces utilisateur

L'interface suit les principes du Material Design 3 avec support des thèmes clair et sombre, une palette personnalisée (vert Talky #25D366) et une navigation par onglets (Conversations, Appels, Statuts).

Table 6.2 – Écrans principaux de l'application Talky

Écran	Description
SplashScreen	Démarrage avec logo animé
OnboardingScreen	Présentation des fonctionnalités (3 slides)
PhoneScreen	Saisie du numéro de téléphone
OtpScreen	Vérification du code OTP
ProfileSetupScreen	Création du profil (nom + photo)
HomeScreen	Hub principal (onglets Chats/Appels/Statuts)
ChatScreen	Interface de messagerie complète
CallScreen	Écran d'appel actif (audio/vidéo)
IncomingCallScreen	Écran d'appel entrant (Accepter/Refuser)
StatusScreen	Liste des statuts des contacts
StatusViewerScreen	Visionneuse de statuts avec barre de progression
SettingsScreen	Paramètres (thème, langue, profil, à propos)

6.4 Sécurité

- Les clés Firebase et Cloudinary sont stockées dans des variables d'environnement (fichier `.env` exclu par `.gitignore`).
- Le service account Firebase Admin est injecté via la variable d'environnement `FIREBASE_SERVICE_ACCOUNT` sur Render.com.
- Les règles Firestore restreignent l'accès aux données aux seuls participants des conversations.
- L'OTP garantit que chaque compte est lié à un numéro de téléphone réel et vérifié.

Chapitre 7

Conclusion et perspectives

7.1 Bilan

Ce projet nous a permis de concevoir et d'implémenter de A à Z une application de messagerie professionnelle en mobilisant un large spectre de technologies modernes. Les principaux livrables sont :

— Une application Android fonctionnelle avec messagerie texte et multimédia complète.

— Un système d'appels audio/vidéo P2P via WebRTC avec prise en charge des appels de groupe.

— Un système de statuts éphémères 24h avec compteur de vues et réactions.

— Un serveur de signalisation Node.js déployé en production sur Render.com.

— Une architecture NoSQL bien structurée avec index Firestore optimisés.

— Un système de notifications push robuste basé sur FCM data-only.

Sur le plan académique, ce projet a renforcé nos compétences en modélisation (MCD/MLD/UML), architecture logicielle, développement mobile multiplateforme et gestion de bases de données NoSQL temps réel.

7.2 Difficultés rencontrées

1. **Notifications FCM sur Android :** la présence du champ `notification` dans le payload FCM provoquait l'interception par Android et rendait inopérante la navigation vers l'écran d'appel. La solution : messages *data-only*.
2. **Écran d'appel entrant :** l'utilisation de `FlutterActivity` comme parent de l'activité Android native générait un second moteur Flutter isolé. Le passage à `AppCompatActivity` a résolu ce problème.
3. **Candidats ICE prématurés :** des candidats ICE étaient émis avant que la connexion peer soit prête, causant des échecs silencieux.

4. **Contraintes Firebase Spark :** Firebase Storage étant limité géographiquement, nous avons adopté Cloudinary comme alternative.

7.3 Perspectives d'évolution

P1 Traduction instantanée : intégration de l'API Google Translate pour traduire les messages dans la langue de l'utilisateur.

P2 Mode confidentiel : messages à auto-destruction avec durée configurable et prévention des captures d'écran.

P3 Version web : portage Flutter Web ou React.js partageant la même infrastructure Firebase.

P4 Chiffrement de bout en bout : implémentation du protocole Signal avant stockage Firestore.

P5 Déploiement iOS : portage et certification App Store.

P6 Tableau de bord d'administration : interface web de gestion et monitoring.

En définitive, **Talky** constitue une démonstration concrète qu'une équipe académique peut, avec les outils modernes disponibles, concevoir et livrer une solution de communication complète et opérationnelle. Ce projet pose les fondations d'un produit qui, avec les évolutions planifiées, pourrait être déployé à l'échelle d'une organisation.

Bibliographie

[1] Google. *Flutter Documentation*. https://docs.flutter.dev, 2024.

[2] Google. *Firebase Documentation — Firestore, FCM, Realtime Database*. https://firebase.google.com/docs, 2024.

[3] W3C / IETF. *WebRTC 1.0 : Real-Time Communication Between Browsers*. https://www.w3.org/TR/webrtc/, 2023.

[4] Socket.IO. *Socket.IO Documentation v4*. https://socket.io/docs/v4/, 2024.

[5] R. Rousselet. *Riverpod Documentation*. https://riverpod.dev, 2024.

[6] Cloudinary. *Upload API Reference*. https://cloudinary.com/documentation, 2024.

[7] H. Tardieu, A. Rochfeld, R. Colletti. *La méthode MERISE — Principes et outils*. Éditions d'Organisation, Paris, 1983.

[8] M. Fowler. *UML Distilled*, 3e éd. Addison-Wesley, 2003.

[9] P. Sadalage, M. Fowler. *NoSQL Distilled*. Addison-Wesley, 2012.

Chapitre A

Structure du projet Flutter

Listing A.1 – Arborescence complète du projet Talky

```
talky/
+-- android/
|   +-- app/src/main/
|       +-- kotlin/.../MainActivity.kt
|       +-- kotlin/.../IncomingCallActivity.kt
|       +-- AndroidManifest.xml
+-- lib/
|   +-- core/
|   |   +-- cache/          local_cache.dart
|   |   +-- constants/      app_constants.dart, app_colors.dart
|   |   +-- router/         app_router.dart
|   |   +-- services/       notification_service.dart
|   |   |                   presence_service.dart
|   |   |                   fcm_sender.dart
|   |   |                   ringback_service.dart
|   |   +-- theme/          app_theme.dart
|   +-- features/
|   |   +-- auth/           phone_screen, otp_screen, register_screen
|   |   +-- chat/           chat_screen, conversations_screen
|   |   |                   message_model, conversation_model
|   |   +-- calls/          call_screen, call_service
|   |   |                   call_history_model
|   |   +-- status/         status_screen, status_model
|   |   |                   status_service
|   |   +-- settings/       settings_screen, appearance
|   |   |                   language_settings
|   |   +-- home/           home_screen
|   |   +-- profile/        profile_setup_screen
|   |   +-- splash/         splash_screen
|   +-- shared/widgets/     talky_button.dart
|   |                       talky_text_field.dart
```

```
|   +-- firebase_options.dart
|   +-- main.dart
+-- server.js           Serveur Node.js / Socket.IO
+-- pubspec.yaml
+-- .env                (non versionne)
```

Chapitre B

Serveur de signalisation — Extraits clés

Listing B.1 – Endpoint FCM data-only

```
app.post('/notify', async (req, res) => {
  const { toUserId, title, body, type,
          conversationId, callerId } = req.body;

  const userDoc = await db.collection('users')
                           .doc(toUserId).get();
  const fcmToken = userDoc.data().fcmToken;

  // Pas de champ "notification" -> data-only
  // Flutter gere l'affichage lui-meme
  const message = {
    token: fcmToken,
    data: {
      type:           String(type  ?? 'message'),
      title:          String(title ?? 'Talky'),
      body:           String(body  ?? ''),
      conversationId: String(conversationId ?? ''),
      callerId:       String(callerId       ?? ''),
    },
    android: {
      priority: type === 'call' ? 'high' : 'normal',
    },
  };
  await admin.messaging().send(message);
  res.json({ success: true });
});
```

Chapitre C

Règles de sécurité Firestore

Listing C.1 – Règles Firestore recommandées

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Profils : lecture publique, ecriture par le proprietaire
    match /users/{userId} {
      allow read:  if request.auth != null;
      allow write: if request.auth.uid == userId;
    }

    // Conversations : acces aux participants uniquement
    match /conversations/{convId} {
      allow read, write: if request.auth.uid in
        resource.data.participantIds;

      match /messages/{msgId} {
        allow read, write: if request.auth.uid in
          get(/databases/$(database)/documents/
              conversations/$(convId))
            .data.participantIds;
      }
    }

    // Statuts : lecture authentifiee, ecriture par auteur
    match /statuses/{statusId} {
      allow read:  if request.auth != null;
      allow write: if request.auth.uid
                   == resource.data.userId;
    }
  }
}
```